



Introduction to Machine Learning

Артеми́й Мосе́йчук

tg: @gvolsky

Ма́рия Митра́нкова

tg: @mitrankova

Based on materials from Alexey Shpilman and Oleg Svidchenko

Оценка

Составляющие: квизы, домашние работы, экзамен

За каждый элемент оценка от 0 до 10

Блокирующие элементы: домашние работы, экзамен

По домашним работам два дедлайна: мягкий и жесткий. После мягкого дедлайна можно получить только половину баллов. После жесткого дедлайна задания не принимаются (отдельные случаи обговариваются с преподавателем).

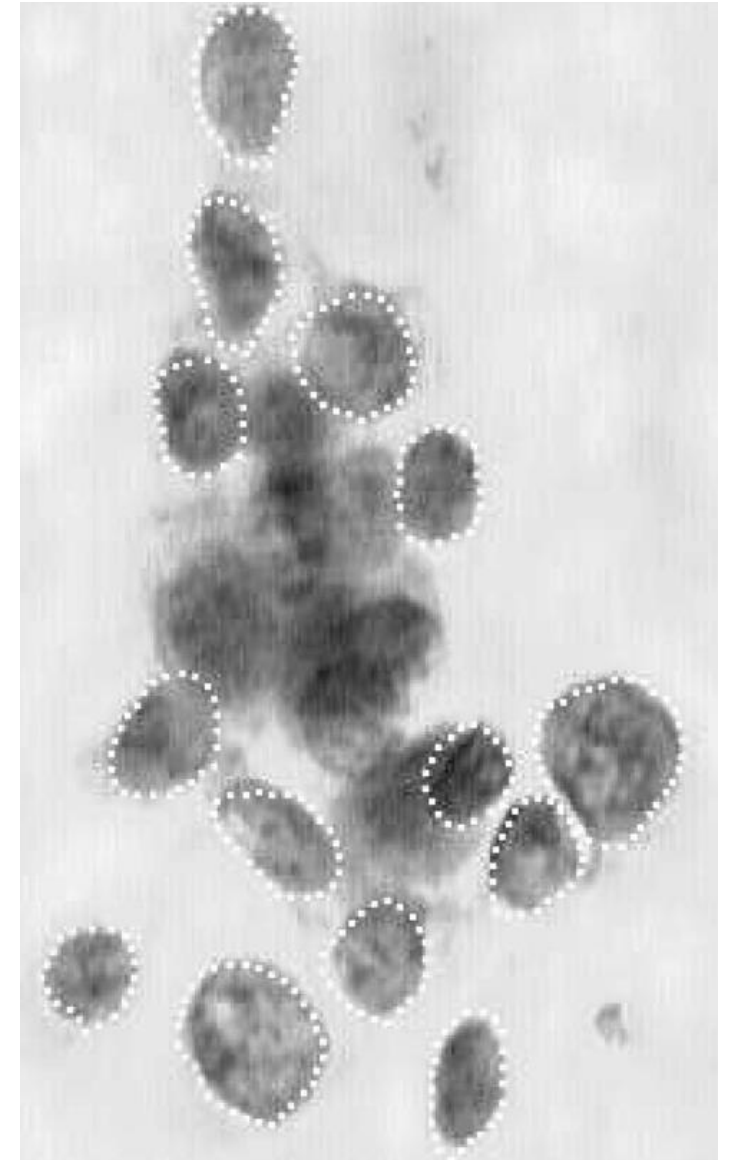
Итог = $0.15 \times \text{квизы} + 0.25 \times \text{домашние работы} + 0.6 \times \text{экзамен}$

Benign/Malignant Tumor Dataset

- 1) ID number
- 2) Diagnosis (M = malignant, B = benign)
- 3-32)

Ten real-valued features are computed for each cell nucleus (mean, highest and standard error):

- a) radius (mean of distances from center to points on the perimeter)
- b) texture (standard deviation of gray-scale values)
- c) perimeter
- d) area
- e) smoothness (local variation in radius lengths)
- f) compactness ($\text{perimeter}^2 / \text{area} - 1.0$)
- g) concavity (severity of concave portions of the contour)
- h) concave points (number of concave portions of the contour)
- i) symmetry
- j) fractal dimension (“coastline approximation” - 1)



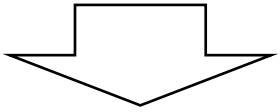
Features

label	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
M	17.99	10.38	122.8	1001	0.1184	0.2776	0.3001	0.1471	0.2419	0.07871	1.095	0.9053	8.589	153.4	0.006399	0.04904	0.05373
M	20.57	17.77	132.9	1326	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	0.5435	0.7339	3.398	74.08	0.005225	0.01308	0.0186
M	19.69	21.25	130	1203	0.1096	0.1599	0.1974	0.1279	0.2069	0.05999	0.7456	0.7869	4.585	94.03	0.00615	0.04006	0.03832
B	13.54	14.36	87.46	566.3	0.09779	0.08129	0.06664	0.04781	0.1885	0.05766	0.2699	0.7886	2.058	23.56	0.008462	0.0146	0.02387
B	13.08	15.71	85.63	520	0.1075	0.127	0.04568	0.0311	0.1967	0.06811	0.1852	0.7477	1.383	14.67	0.004097	0.01898	0.01698
B	9.504	12.44	60.34	273.9	0.1024	0.06492	0.02956	0.02076	0.1815	0.06905	0.2773	0.9768	1.909	15.7	0.009606	0.01432	0.01985
M	15.34	14.26	102.5	704.4	0.1073	0.2135	0.2077	0.09756	0.2521	0.07032	0.4388	0.7096	3.384	44.91	0.006789	0.05328	0.06446
M	13	21.82	87.5	519.8	0.1273	0.1932	0.1859	0.09353	0.235	0.07389	0.3063	1.002	2.406	24.32	0.005731	0.03502	0.03553
M	12.46	24.04	83.97	475.9	0.1186	0.2396	0.2273	0.08543	0.203	0.08243	0.2976	1.599	2.039	23.94	0.007149	0.07217	0.07743
B	12.05	14.63	78.04	449.3	0.1031	0.09092	0.06592	0.02749	0.1675	0.06043	0.2636	0.7294	1.848	19.87	0.005488	0.01427	0.02322
B	13.49	22.3	86.91	561	0.08752	0.07698	0.04751	0.03384	0.1809	0.05718	0.2338	1.353	1.735	20.2	0.004455	0.01382	0.02095
B	11.76	21.6	74.72	427.9	0.08637	0.04966	0.01657	0.01115	0.1495	0.05888	0.4062	1.21	2.635	28.47	0.005857	0.009758	0.01168
B	13.64	16.34	87.21	571.8	0.07685	0.06059	0.01857	0.01723	0.1353	0.05953	0.1872	0.9234	1.449	14.55	0.004477	0.01177	0.01079
B	11.94	18.24	75.71	437.6	0.08261	0.04751	0.01972	0.01349	0.1868	0.0611	0.2273	0.6329	1.52	17.47	0.00721	0.00838	0.01311
M	14.54	27.54	96.73	658.8	0.1139	0.1595	0.1639	0.07364	0.2303	0.07077	0.37	1.033	2.879	32.55	0.005607	0.0424	0.04741
M	14.68	20.13	94.74	684.5	0.09867	0.072	0.07395	0.05259	0.1586	0.05922	0.4727	1.24	3.195	45.4	0.005718	0.01162	0.01998
M	16.13	20.68	108.1	798.8	0.117	0.2022	0.1722	0.1028	0.2164	0.07356	0.5692	1.073	3.854	54.18	0.007026	0.02501	0.03188

Supervised Learning

Formalization:

- Input: $x \in X$ (vector of numerical characteristics of a tumor)
- Output (label): $y \in Y$ (benign/malignant tumor?)
- Target: $f: X \rightarrow Y$ (ideal formula that knows the type of any tumor)
- Data (D): $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_N, y_N)$ (collected/historical data)



- Hypothesis: $g: X \rightarrow Y$ (formula to be used, ideally as similar to f as possible)

Classification problem – y belongs to a set (of classes)

Regression problem – y is a real valued number (or a vector)

y_i	x_i												
B	11.94	18.24	75.71	437.6	0.08261	0.04751	0.01972	0.01349	0.1868	0.0611	0.2273	0.6329	
M	14.54	27.54	96.73	658.8	0.1139	0.1595	0.1639	0.07364	0.2303	0.07077	0.37	1.033	

Instance Based Learning or Lazy Learning

“A man is known by the company he keeps”

Similarity-based classifier:

$$h(x; D) = \arg \max_{y \in Y} \underbrace{\sum_{x_i \in D} [y_i = y] w(x_i, x)}_{\Gamma_y(x)}$$

$w(x_i, x)$ – weight of x_i for x

$\Gamma_y(x)$ – affinity of x to class y

For similarity-based classifier we need only **semimetric** on set of X .

kNN – k-Nearest Neighbors

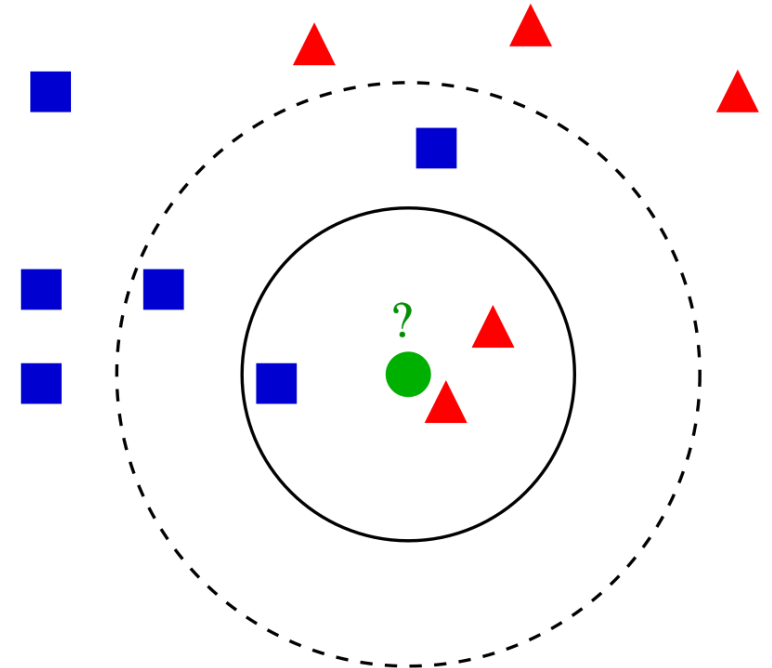
$$h(x; D) = \arg \max_{y \in Y} \sum_{x_i \in D} [y_i = y] w(x_i, x)$$

$w(x_i, x) = 1$, if x_i – one of the k nearest neighbors of x

$w(x_i, x) = 1$, if distance $\rho(x_i, x) < R$ (Radius Neighbors)

Examples of the ρ

- Minkowski: $\rho(x, y) = (\sum_i |x_i - y_i|^p)^{\frac{1}{p}}$
- Jaccard: $\rho(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$
- Levenshtein



kNN: how to choose k?

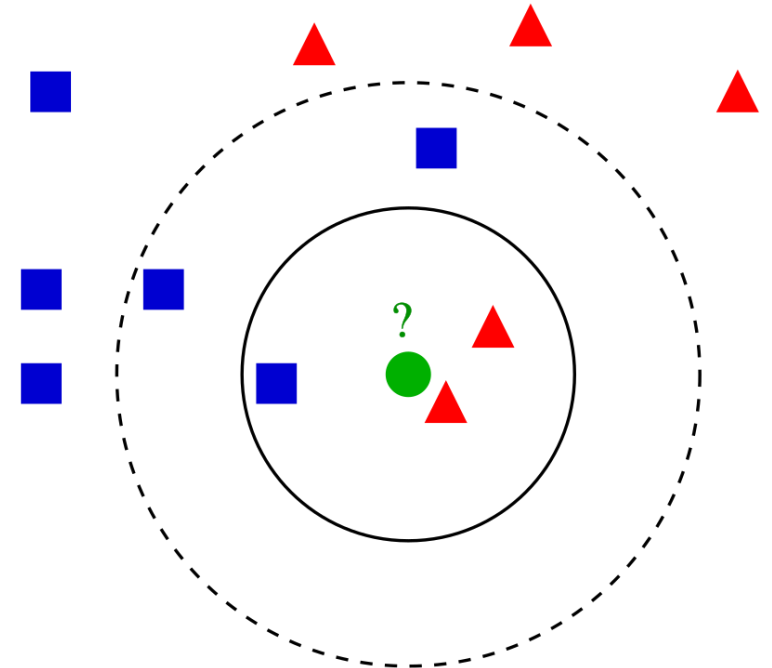
$$h(x; D) = \arg \max_{y \in Y} \sum_{x_i \in D} [y_i = y] w(x_i, x)$$

$w(x_i, x) = 1$, if x_i – one of the k nearest neighbors of x

$w(x_i, x) = 1$, if distance $\rho(x_i, x) < R$ (Radius Neighbors)

Leave-One-Out error:

$$LOO(k, D) = \frac{\sum_{x_i \in D} [h_k(x_i; D / \{x_i\}) \neq y_i]}{|D|} \rightarrow \min_k$$



Train/Test

Put aside a part of a dataset (for example 10 – 30% of all data) for testing (**test dataset**).

Train classifier on what is left (**training dataset**).

Optimize hyperparameters, such as the number of neighbors, looking at the accuracy on the test dataset.

If we have too many hyperparameters, there is a present danger of overfitting on **test dataset**. Put aside one more dataset (**validation dataset**).

Train/Validate/Test

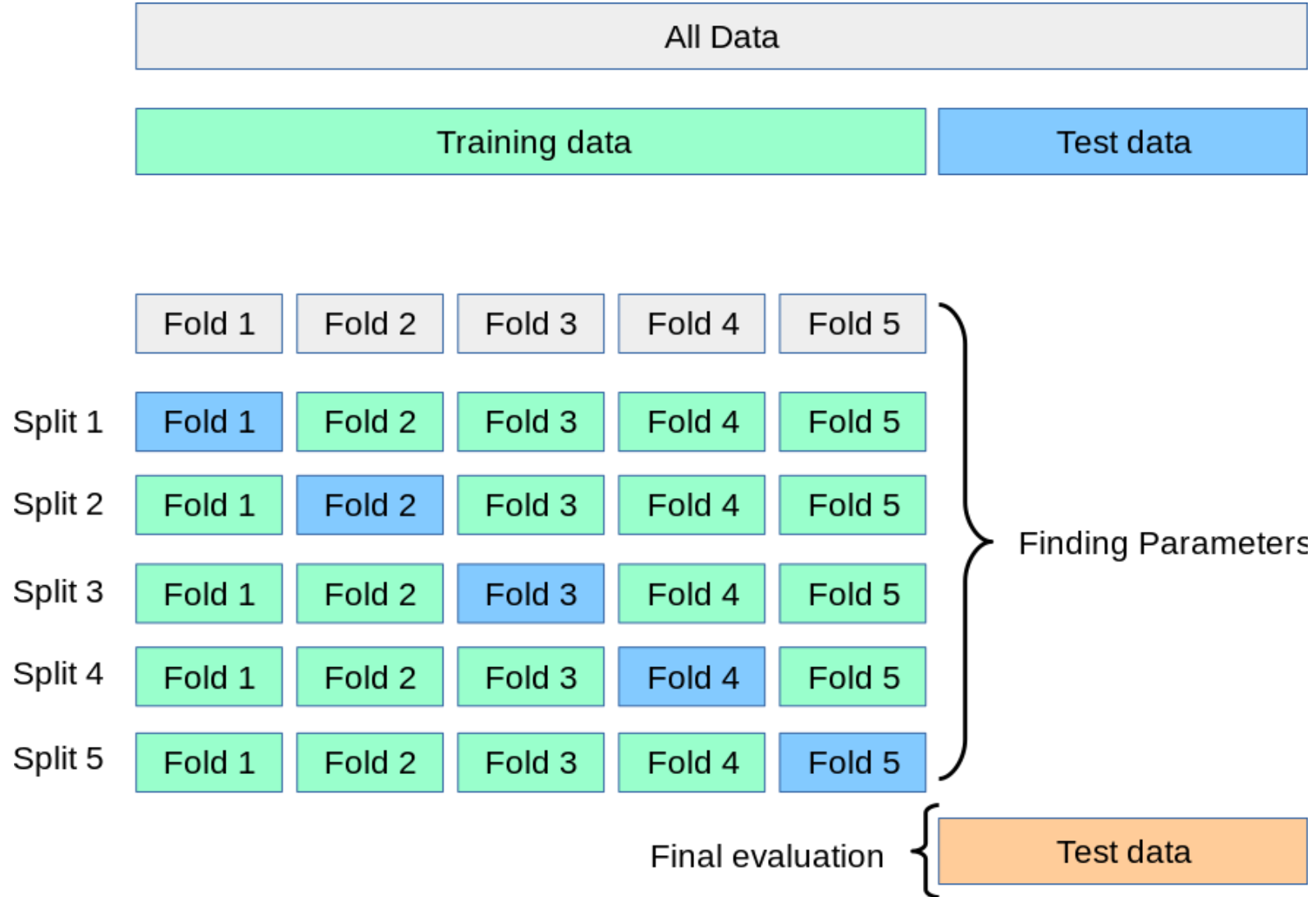
Put aside a part of a dataset (for example 10 – 30% of all data) for validation (**validation dataset**) and a part for testing (**test dataset**).

Train classifier on what is left (**training dataset**).

Optimize hyperparameters, such as the number of neighbors, looking at the accuracy on the **validation dataset**.

Compare classifiers on the **test dataset**.

Cross-Validation

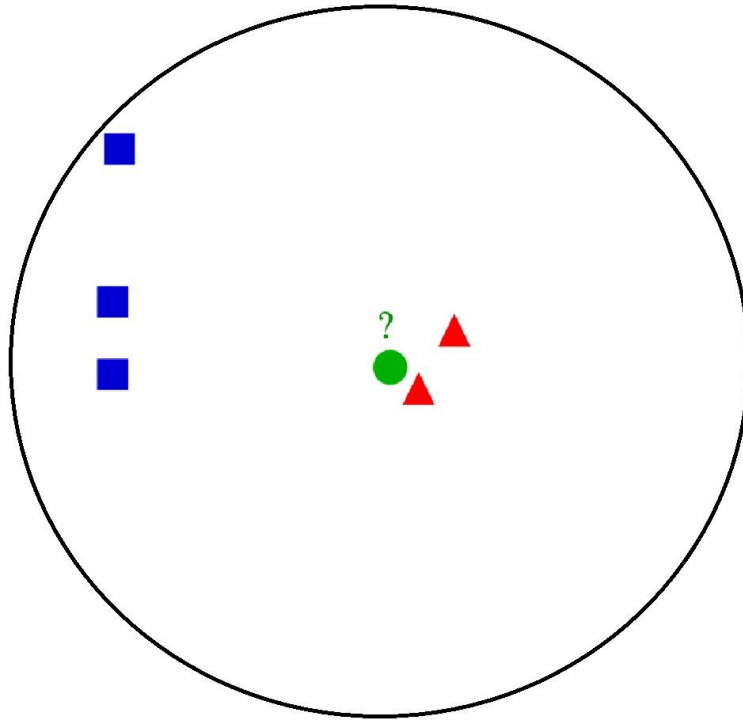


kNN: problems

$$h(x; D) = \arg \max_{y \in Y} \sum_{x_i \in D} [y_i = y] w(x_i, x)$$

$w(x_i, x) = 1$, if x_i – one of the k nearest neighbors of x

$w(x_i, x) = 1$, if distance $\rho(x_i, x) < R$ (Radius Neighbors)



WkNN – weighted kNN

Properties of weight function

- Non negative
- Non increasing

Examples of functions for w

- Triangle kernel: $w_i(x, x_i) = \left[\frac{r - \rho(x, x_i)}{r} \right]_+$
- Gaussian kernel: $w_i = e^{-2\rho(x, x_i)^2}$

WkNN: Parzen Windows

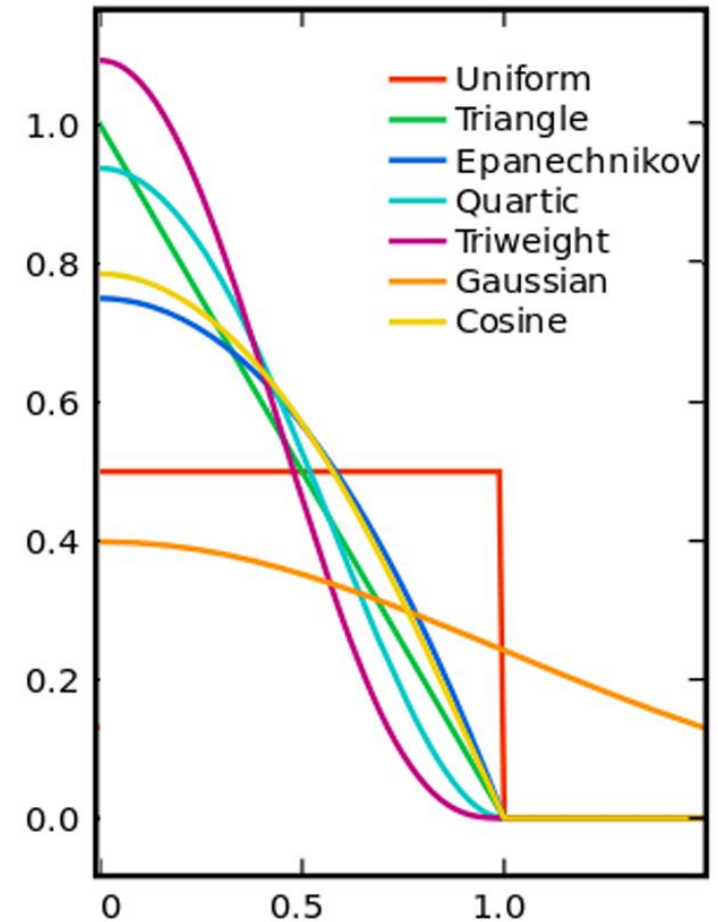
Examples of functions for w

- Triangle kernel: $w_i(x, x_i) = \left[\frac{r - \rho(x, x_i)}{r} \right]_+$
- Gaussian kernel: $w_i = e^{-2\rho(x, x_i)^2}$

Parzen window method

$K: \mathbb{R} \rightarrow \mathbb{R}_+$ (have properties of the w function)

- Fixed width: $w(x, x_i) = K\left(\frac{\rho(x, x_i)}{r}\right)$
- kNN width: $w(x, x_i) = K\left(\frac{\rho(x, x_i)}{\rho(x, x_j)}\right)$, where x_j is the $(k + 1)st$ neighbor



kNN: Properties

Computational complexity

- Space complexity is $O(n)$ (kNN is a non-parametric method)
- Time complexity is also $O(n)$ in base implementation

Can be modified to output predicted probabilities: $P(y_i|x) = \frac{\sum_{l=1}^k [y_l=y_i]}{k}$

Can be adapted for regression problem: the average instead the majority vote

kNN does not behave well in high dimensions (Curse of Dimensionality in the next series)